CENG-567
REINFORCEMENT LEARNING

Lecture-9

Deep Q-Learning

Dr. Osman GÖKALP

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

Monte Carlo (MC) Methods

Core idea: Learn from *complete episodes* without needing a model (model-free).

Prediction

Estimate values by averaging *returns* from sampled episodes.

First-visit MC: update value only on the first visit of (s) or (s, a).

Control

Use **Generalized Policy Iteration (GPI)**:
Policy Evaluation ⇆ Policy Improvement

On-policy MC control:

- Estimate **q(s, a)**.
- Use **ε-greedy** for exploration.
- Improve policy after each episode.

Off-policy MC control:

- Learn about a *target* policy using data from a *behavior* policy.
- Uses **importance sampling**
  - Ordinary IS: unbiased but high variance
  - Weighted IS: low variance (preferred)

Recap

Importance Sampling

Goal: Use data collected from one distribution (behavior policy) to evaluate another (target policy).

Importance Sampling Ratio

$$\rho = \frac{\pi(a \mid s)}{b(a \mid s)}$$

Why it works

Trajectory probabilities differ only in policy terms.

Environment transitions cancel out.

Use in Off-policy MC

Adjust returns using cumulative ρ

Weighted IS reduces variance:

$$v_\pi(s) = \frac{\sum \rho \ G}{\sum \rho}$$

Takeaways

Essential for off-policy learning.

Used when the target policy is *deterministic* but behavior policy must be *soft*.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Recap

Temporal Difference (TD) Learning

Core idea: Update values *after each step* using bootstrapping.

TD(0) Prediction

$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$$

No need to wait for episode end.

Lower variance than MC, but slightly biased due to bootstrapping.

On-policy Control — SARSA

$$\hat{q}_\pi(s,a) := \hat{q}_\pi(s,a) + \alpha[r + \gamma \hat{q}_\pi(s',a') - \hat{q}_\pi(s,a)]$$

Uses ε-greedy action *actually taken*.

Converges in the limit; in practice near-optimal.

Off-policy Control — Q-learning

$$\hat{q}_\pi(s,a) := \hat{q}_\pi(s,a) + \alpha[r + \gamma \max_u \hat{q}_\pi(s',u) - \hat{q}_\pi(s,a)]$$

Updates using the *greedy* action.

Exploratory actions do **not** affect value estimates.

Very widely used.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Motivation

Why Move Beyond Dynamic Programming (DP)?

DP requires **complete knowledge** of the environment's reward and transition dynamics

DP uses **tabular state–action representations**, which **do not scale** when the state space is large

These two limitations motivated the shift to new methods

MC and TD as Solutions

Monte Carlo (MC) and Temporal Difference (TD) methods learn from **interaction with the environment**

They **do not require knowing** the environment dynamics

But: MC and TD **still rely on tabular representations**, so scalability remains an issue

Deep Reinforcement Learning (Deep RL)

Deep RL uses neural networks' **representational power** to learn policies in many situations

However, using function approximators removes many **theoretical guarantees** that tabular Q-learning had

Much of deep Q-learning's success comes from **stabilizing tricks** that make neural networks work effectively for RL

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

From tabular Q-learning to deep Q-learning

Why Tabular Q-Learning Doesn't Scale?

Tabular methods (dynamic prog., MC, TD) **cannot extend** to most real-life RL tasks

Example: an RL agent using **128 × 128 RGB images** has $256^{128\times128\times3}$ possible inputs — astronomically large

Therefore, we need **function approximators** to represent value functions

Why Deep Learning?

Neural networks have proven highly successful in **supervised** and **unsupervised** learning

They provide the **representational power** needed for complex RL inputs

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

From tabular Q-learning to deep Q-learning

But There's a Challenge

The **theoretical guarantees** of tabular Q-learning **break down** when using function approximators

This makes stability and convergence more difficult

What This Lecture Covers

Two deep Q-learning approaches:

- **Neural-Fitted Q-Iteration (NFQ)**
- **Online Q-Learning**

Discussion of the **issues and limitations** they face

Prepares the foundation for **modern deep Q-learning methods** in the next sections

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Neural-fitted Q-iteration

NFQ trains a neural network to represent **action values (the Q-function)**

The network is fitted to **target Q-values** that are:

- **sampled from the environment**, and
- **bootstrapped** using previously available Q-values

Recap: Tabular Q-Learning Update

In tabular Q-learning, action values are learned from **environment samples**

Each sample is a tuple:

$$(s, a, r, s')$$

Learning occurs by repeatedly applying the **Q-learning update rule**

$$\mathrm{Q(s,a)} := \mathrm{Q(s,a)} + \alpha[\mathrm{r} + \gamma \max_{\mathrm{a}'} \mathrm{Q}(s', \mathrm{a}') - \mathrm{Q(s,a)}]$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Neural-fitted Q-iteration

Q represents an estimate of the action values of the optimal policy, q^*. (Deep RL convention uses **capital Q**)

Goal: update the estimate $\mathrm{Q}(\mathrm{s}, \mathrm{a})$ toward the target

$$y = \mathrm{r} + \gamma \max_{\mathrm{a}'} \mathrm{Q}(s', \mathrm{a}')$$

by applying the sampled Bellman optimality operator to the sample (s, a, r, s')

How NFQ Differs?

Q-values are represented by a **neural network** with parameters θ, written as Q_θ ,instead of a table

NFQ uses a **batch** of samples and fits the network to the target values **all at once**, rather than updating incrementally

NFQ performs **multiple rounds** of:

- computing targets using the current Q-function
- fitting the network parameters so the targets improve as the Q-function improves

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

NFQ Algorithm

1. Initialize θ and a policy, π.
2. Collect a set of N samples, $\{(s_i, a_i, r_i, s_i') \mid i = 1, \ldots, N\}$, using the π policy.
3. Apply the sampled Bellman optimality operator to obtain the target values, $y_i = r_i + \gamma \max_{a_i'} Q_\theta(s_i', a_i')$, to all samples, $i = 1, \ldots, N$, but if s_i' is a terminal state, set $y_i = r_i$.
4. Obtain θ by minimizing the gap between Q_θ and the target values. More formally, $\theta := \arg\min_\theta \sum_{i=1}^{N} L(Q_\theta(s_i, a_i), y_i)$, where L is a loss function, such as squared error, $\frac{1}{2}(Q_\theta(s_i, a_i) - y_i)^2$.
5. Update π with respect to the new Q_θ value.

** Loop between steps 2 and 5 until convergence*

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Practical Considerations for Fitted Q-Iteration

Use a **soft policy** (e.g., ε-greedy) during sample collection to ensure sufficient exploration.

Too large ε can prevent reaching states that require following a good policy for many steps (e.g., later levels in games).

Target values may be inaccurate because they bootstrap from inaccurate $Q_\theta(s_i', a_i')$.

Therefore, **repeat the target computation and fitting steps K times**.

The **initial policy** may not reach all parts of the state space.

Collect **more samples after each policy update** and add them to the dataset.

The method is **off-policy**: samples may come from the current policy or from another controller.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Challenges with Fitted Q-Iteration

- Each time new target values are computed, the algorithm must **train** θ **from scratch in Step 4**.
  - Step 4 involves an **arg min** θ optimization over the entire target batch.
  - Unlike incremental gradient descent, parameters are **not updated gradually** with new samples.
  - In large-scale RL (billions of samples), repeatedly retraining the network with updated targets becomes **impractical**.
- The convergence guarantees of tabular SARSA and Q-learning **do not hold** when function approximators are introduced.
- Using function approximation + Q-learning (off-policy) + bootstrapping is especially unstable — known as the **deadly triad**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Online Q-Learning

- **Limitation of Fitted Q-Iteration**
  - Requires solving **arg min θ** from scratch for every new batch of samples
  - Computation becomes **impractical** when data is large or the problem is complex
- **Online Q-Learning Idea**
  - Takes a **single gradient update** to θ after **each observed transition** (s_i, a_i, r_i, s_i')
  - Completely eliminates repeated full batch optimization
- **Key Advantage**
  - **Gradual Updates with incremental learning**
  - Updates Q-values continuously while interacting with the environment

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Online Q-Learning

1. Initialize $\boldsymbol{\theta_0}$ and a policy, $\boldsymbol{\pi_0}$, then initialize the environment and observe $\boldsymbol{s_0}$.

   For $t = 1$ to T, continue with the following steps.

2. Take some action, $\boldsymbol{a_{t-1}}$, using a policy, $\boldsymbol{\pi_{t-1}}$, given the state, $\boldsymbol{s_{t-1}}$, then observe $\boldsymbol{r_t}$ and $\boldsymbol{s_t}$, which form the $(\boldsymbol{s_{t-1}}, \boldsymbol{a_{t-1}}, \boldsymbol{r_t}, \boldsymbol{s_t})$ tuple.

3. Obtain the target value, $\boldsymbol{y_t} = \boldsymbol{r_t} + \boldsymbol{\gamma} \max_{\boldsymbol{a}} \boldsymbol{Q_{\theta_{t-1}}}(\mathbf{s_t}, \boldsymbol{a})$, but if $\mathbf{s_t}$ is a terminal state, set $\boldsymbol{y_t} = \boldsymbol{r_t}$.

4. Take a gradient step to update $\boldsymbol{\theta_t} := \boldsymbol{\theta_{t-1}} - \boldsymbol{\alpha} \boldsymbol{\nabla_\theta} \boldsymbol{L}(\boldsymbol{Q_{\theta_{t-1}}}(\boldsymbol{s_{t-1}}, \boldsymbol{a_{t-1}}), \boldsymbol{y_t})$, where $\boldsymbol{\alpha}$ is the step size.

5. Update the policy to $\boldsymbol{\pi_t}$ with respect to the new $\boldsymbol{Q_{\theta_t}}$ value.

   End for

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Online Q-Learning

- Similar to the NFQ algorithm, we **need** a policy that continuously **explores the state-action space**. Again, this can be achieved by using **an ϵ-greedy** policy or another **soft policy**.

- Also similar to fitted Q-iteration, the **samples may come from a policy that is not relevant to what the Q-network is suggesting**, as this is an **off-policy** method.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Online Q-Learning

Challenges

- **The gradient estimates are noisy**
  - it uses a single sample, which results in noisy estimates that make it hard to optimize the loss function
- **The gradient step is not truly gradient descent**
  - we treat y_t as a constant even though it is not. y_t itself depends on θ, yet we ignore this fact by not taking its derivative with respect to θ.
- **Target values are updated after each gradient step, which becomes a moving target that the network is trying to learn from**
  - In supervised learning labels don't change, but here it changes
- **The samples are not independent and identically distributed (i.i.d.)**
  - Samples are highly correlated (MDP is a sequential decision). Another deviation from the classical gradient descent in supervised learning.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Deep Q-networks

- The **DQN** is a seminal work by Mnih et al. (2015) that made deep RL a **viable approach to complex sequential control problems**.

- The authors demonstrated that a **single DQN architecture can achieve super-human-level performance in many Atari games** without any feature engineering, which created a lot of excitement regarding the progress of AI.

- In fact, it is with the DQN that we started talking about **deep RL**.

Key concepts in DQNs (Experience Replay)

- **Problem:** Sequential samples from the environment are *highly correlated* → leads to unstable gradient updates in online Q-learning.
- **Solution:**
  Store transitions (s, a, r, s_i')in a **replay buffer** (Lin, 1993).
  Sample ***minibatches*** **uniformly at random**:
  - Breaks temporal correlations
  - Produces i.i.d.-like training data
  - Stabilizes neural network training
- **Key Benefits:**
  - **Decorrelated samples** → more stable learning
  - **Reuses past experience** instead of discarding it
  - **Reduces required environment interaction** (critical when data is expensive)

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Key concepts in DQNs (Target Networks)

Problem:
Bootstrapping with neural networks creates **moving targets**.
Learning becomes unstable because the target y changes whenever θ changes.

Solution:
Maintain a **separate target network** with parameters θ'.
Use it **only** to compute target values:

$$y_j = r_j + \gamma \max_{a_j'} Q_{\theta'}(s_j', a_j')$$

Update Rule:
Every C steps, copy parameters from the main network:

$$\theta' \leftarrow \theta$$

Effect:

- Targets become **much more stable**
- Main network can learn effectively
- Slight staleness is acceptable and improves convergence

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The loss function

- With experience replay and the target network introduced, the **DQN minimizes the following loss function**:

$$\mathrm{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim \mathrm{U(D)}}[(\mathrm{r} + \gamma \max_{a'} Q_{\theta'}(s', a') - Q_{\theta}(s, a))^2]$$

- Here, D **is the replay buffer**, from which a **minibatch** of (s, a, r, s') tuples are **drawn uniformly at random** to update the neural network.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The DQN algorithm (pseudocode)

1. Initialize θ and a replay buffer, D, with a fixed capacity, M. Set the target network parameters as $\theta' := \theta$.
2. Set the policy, π, to be ϵ-greedy with respect to Q_θ.
3. Given the state, s, and the policy, π, take an action, a, and observe r and s'. Add the transition, (s, a, r, s'), to the replay buffer, D. If $|D| > M$, eject the oldest transition from the buffer.
4. If $|D| \geq N$, uniformly sample a random minibatch of N transitions from D, else return to *step 2*.
5. Obtain the target values, $y_j = r_j + \gamma \max_{a'_j} Q_{\theta'}(s'_j, a'_j)$, $j = 1, \ldots, N$ except if s'_j is a terminal state, set $y_j = r_j$.
6. Take a gradient step to update θ, which is $\theta := \theta - \alpha \nabla_\theta J(\theta)$. Here,

$$J(\theta) = \frac{1}{N}\sum_{j=1}^{N} L\big(Q_\theta(s_j, a_j), y_j\big) = \frac{1}{N}\sum_{j=1}^{N}\big(Q_\theta(s_j, a_j) - y_j\big)^2$$

7. Every C steps, update the target network parameters, $\theta' := \theta$.
8. Return to *step* 2

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The DQN algorithm (Overview)

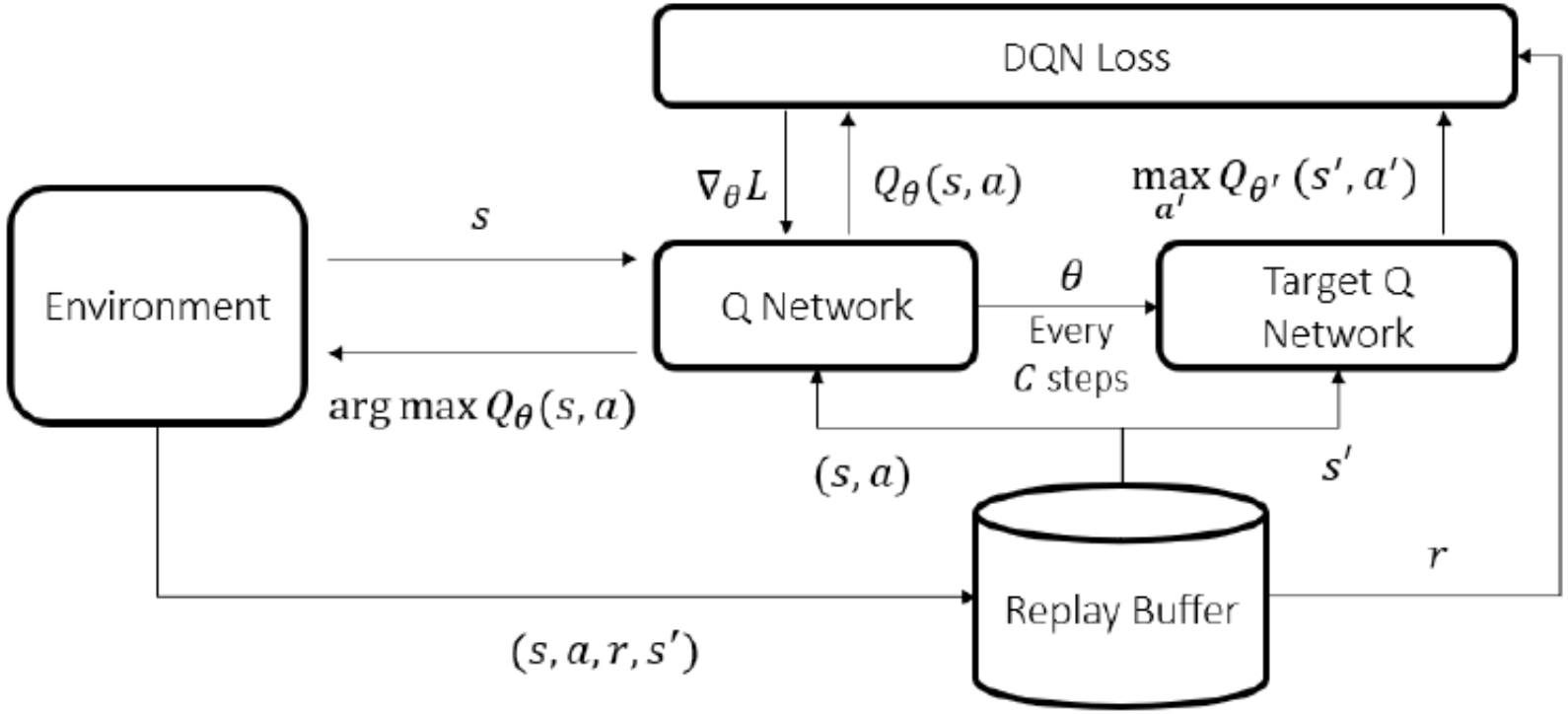


Figure 6.2 – DQN algorithm overview (source: Nair et al., 2015)

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Extensions to the DQN – Rainbow

- After the seminal work on DQNs, there have been many **extensions** proposed to **improve** them in various papers. Hessel et al. (2018) combined some of the most important of those and called them **Rainbow**.
- The Rainbow improvements bring in a significant performance **boost over the vanilla DQN** and they **have become standard** in most **Q-learning implementations**.
- There are **six extensions** to the DQN included in the Rainbow algorithm. These are:

  (i) double Q-learning,
  (ii) prioritized replay,
  (iii) dueling networks,
  (iv) multi-step learning,
  (v) distributional RL,
  (vi) noisy nets.

Extension 1: Double Q-learning

- **Maximization Bias in Q-Learning**

- When computing targets, Q-learning uses:

$$\max_{a} Q_{\theta}(s', a)$$

- **Problem:** Q-value estimates are noisy during learning.

- Maximizing over noisy estimates leads to **overestimation**:
  - We select the **maximum of noisy samples**, not the maximum of true values.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Extension 1: Double Q-learning

- Figure show the probability distributions of obtaining various Q-value estimates for the available actions for a **given state, s**, where the **vertical lines** correspond to the **true action values**.
- **In *(a)***, there are three available actions Just by chance, the estimates we obtain happen to be $Q(s, a_2) > Q(s, a_3) > Q(s, a_1)$. Not only is the best action incorrectly predicted as a_2 but its action value is overestimated.
- **In (b)**, there are six available actions with the same probability distribution of Q-value estimates. Although their true action values are the same, when we take a random sample, an order will appear between them just by chance.
- As a result, Q-value is overestimated.

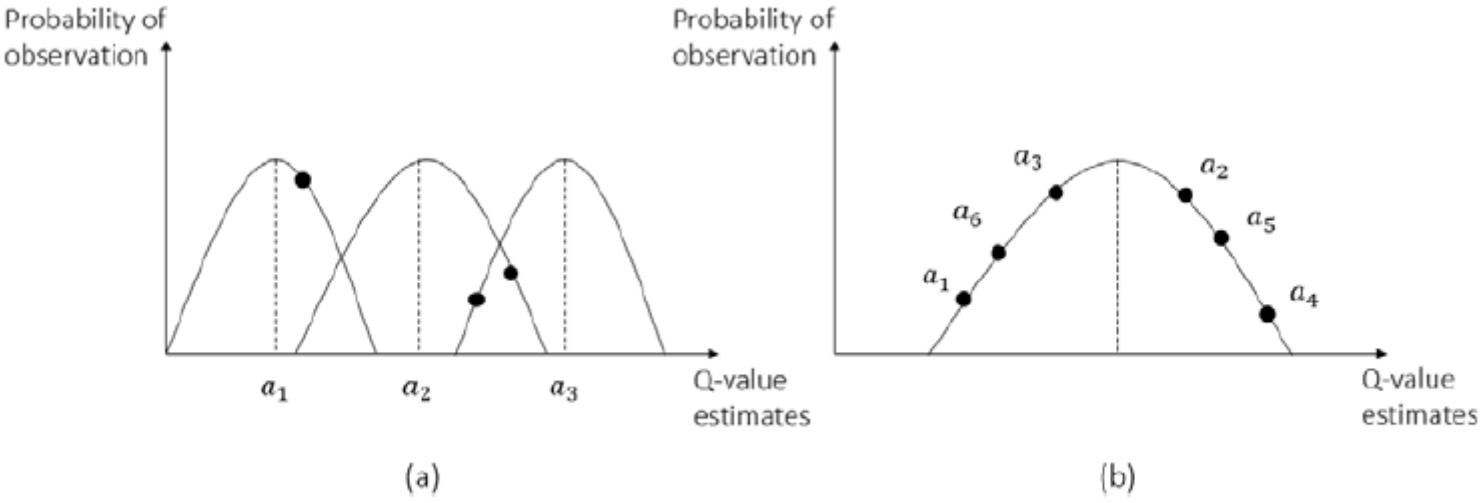


Figure 6.3 – Two examples of maximization bias

Extension 1: Double Q-learning

- Double Q-learning proposes a solution to the maximization bias by **decoupling** finding the **maximizing** action and **obtaining** an action-value **estimate** for it by using **two separate action-value functions, $\boldsymbol{Q_1}$ and $\boldsymbol{Q_2}$.**
- DQN **already** proposes **maintaining a target network** with θ' parameters dedicated to providing action-value estimates for bootstraping. **Therefore**, we implement double Q-learning **on top of the DQN** to obtain the **action-value estimate** for the maximizing action, as follows:

$$Q'_\theta(\mathrm{s}, \arg\max_a Q_\theta(s, a))$$

- Then, the corresponding **loss function** for the state-action pair (s_t, a_t) becomes the following:

$$(r_{t+1} + \gamma Q'_\theta(\mathrm{s}, \arg\max_a Q_\theta(s_t, a)) - Q_\theta(s_t, a_t))^2$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Extension 2: Prioritized Replay

Uniform sampling (as in standard DQN) treats all experiences equally.

But some transitions are more **informative** or **surprising**, especially in:

- Sparse-reward tasks
- Hard-exploration problems
- Many “boring failures’’ vs. few “useful successes’’

Idea (Schaul et al., 2015): Use **TD error** to measure how “interesting’’ an experience is.

Sampling probability:

$$p_t \propto |\delta_t|^{\omega}$$

- δ_t :TD error of experience at time t
- ω: controls how strongly sampling favors high TD error

Effect of ω:

- $\omega = 0$: uniform sampling (original DQN)
- Larger ω: stronger focus on surprising, high-TD-error experiences

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Extension 3: Dueling Networks

Why Dueling Networks?

In some states, actions have **little or no effect** on the environment.
Example:

- A robot enters a **trap state**
- Actions cannot escape
- Only a small random chance moves it out
- Reward is negative while inside the trap

In such states:

- Estimating individual **action-values** $Q(s, a)$ is meaningless
- The agent only needs to know the **state value** $V(s)$
- Forcing the network to predict full Q-values adds noise and hurts performance

Problem: DQN tries to learn meaningful Q(s, a) for actions that do not matter.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Extension 4: Dueling Networks

Dueling Network Architecture

Solution: Learn two quantities in parallel:

State Value: $V(s)$ –how good the state is

Action Advantage:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

Interpretation:
Advantage = extra return gained by choosing action a instead of what the policy recommends.

Key idea:
If actions don't matter in a state (e.g., trap), advantages ≈ 0 → Network focuses on learning **state value** instead.

Choosing actions:
Selecting the action with highest advantage is equivalent to selecting the action with highest Q-value.

With this architecture, the authors obtained state-of-the-art results at the time on the Atari benchmarks, proving the value of the approach!

Extension 4: Dueling Networks

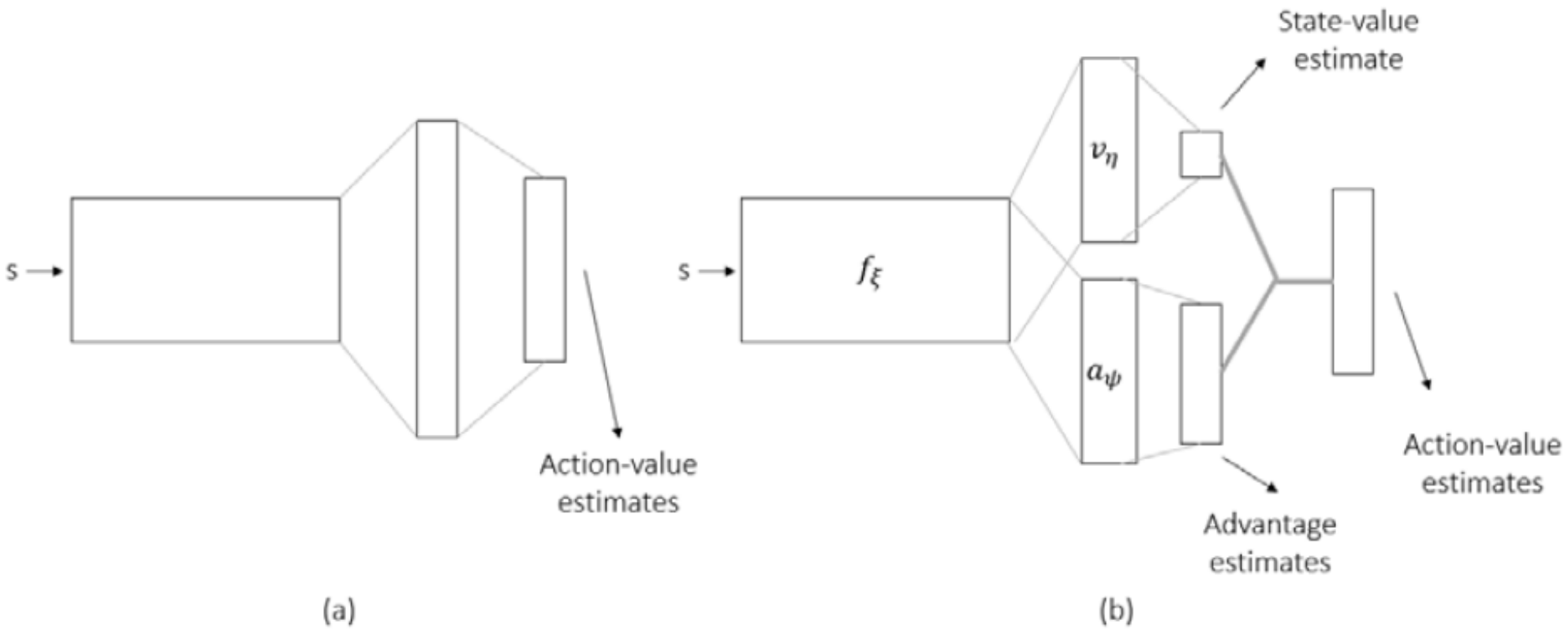


Figure 6.4 – (a) regular DQN and (b) dueling DQN (source: Wang et al., 2016)

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Extension 4: Multi-step Learning

- A **more accurate target value** for a state-action pair can be obtained by **using multi-step discounted** rewards in the estimation obtained from the environment.
- In such cases, the Q-value estimates used in bootstrapping will be **discounted more heavily**, diminishing the impact of the inaccuracies of those estimates.
- **Instead**, more of the **target value will come** from the **sampled rewards**.
- TD error in multi-step setting:

$$R_t^{(n)} \equiv \sum_{k=0}^{n-1} \gamma^k r_{t+k+1}$$

$$\delta_t^{(n)} = R_t^{(n)} + \gamma^n \underset{a'}{\operatorname{argmax}}\, Q_{\theta'}(S_{t+n}, a') - Q_\theta(S_t, a_t)$$

- You can see that with increasing n , the impact of the $\boldsymbol{\underset{a'}{argmax}\, Q_{\theta'}(S_{t+n}, a')}$ term diminishes since $0 < \gamma < 1$.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Extension 5: Distributional Reinforcement Learning

Traditional Q-learning estimates **only the expected return** $\mathbb{E}[G_t]$.

Distributional RL (Bellemare et al., 2017) instead learns the **full probability distribution** of returns.

Value Distribution Representation

Represent the value distribution using **N discrete atoms** over a support vector $\mathbf{z}$.

The network outputs a probability mass function:

$$p_\theta^i(s, a) \textit{ for each atom } z_i$$

Example

If the value range is between

$$v_{\min} = 0, v_{\max} = 100$$

and we use **11 atoms**, then:

$$\mathbf{z} = \{0, 10, 20, \ldots, 100\}$$

- The value network then estimates, for a given s value, what the probability is that its value is 0, 10, 20, and so on. v_{min}, $v_{\max}$, and $\mathrm{N}_{\mathrm{atoms}}$ are additional hyperparameters to be tuned.

Extension 6: Noisy Nets

Problem

Standard Q-learning uses a fixed **ε-greedy** exploration rate.

But **different states may require different levels of exploration**.

Hard-exploration tasks especially suffer from globally fixed ε.

Idea

Replace deterministic linear layers with **parameterized noisy layers**, where the *amount of noise is learned* during training.

Standard Linear Layer

$$y = Wx + b$$

Noisy Linear Layer

$$y = (\mu_W + \sigma_W \odot \epsilon_W)x + (\mu_b + \sigma_b \odot \epsilon_b)$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Extension 6: Noisy Nets

μ_W, μ_b :learned mean parameters

σ_W, σ_b :learned noise scales

ϵ_W, ϵ_b :fixed-distribution noise variables

$\odot$: element-wise multiplication

Benefits

Exploration becomes **state-dependent** and **learned**, not manually tuned.

Replaces ε-greedy entirely in many implementations.

Especially effective in **sparse-reward / hard-exploration** problems (Fortunato et al., 2017).

Performance of the Rainbow Agent

Key Idea

Rainbow combines **all major DQN improvements—**
Double Q-learning, Prioritized Replay, Dueling Networks, Multi-step Returns, Distributional RL, and Noisy Nets—
into **one unified agent**.

Outcome

Achieved **state-of-the-art performance** on the **Atari 2600** benchmark suite.

Demonstrated the power of integrating multiple enhancements rather than using them in isolation.

Key Question

How much does each component contribute?
The Rainbow paper evaluates this via **ablation studies**, measuring performance with and without each improvement.

How to choose which extensions to use – ablations to Rainbow

Rainbow: Importance of Individual Extensions

Key Findings from the Rainbow Ablation Study

Most important extensions

- **Prioritized Experience Replay**
- **Multi-step Learning**
  Removing either caused the **largest performance drop**.

Next most significant

- **Distributional DQN**
  Its benefit becomes more pronounced **later in training**.

Moderately valuable

- **Noisy Nets**
  Removing them decreases performance, but **less dramatically**.

Minimal impact

- **Dueling Architecture**
- **Double Q-Learning**
  Their removal showed **no notable performance change** in Atari benchmarks.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

What Happened to the Deadly Triad?

The Deadly Triad

Off-policy learning + bootstrapping + function approximation
⟶ *theoretically leads to divergence*.

But DQN works well—why?

Hasselt et al. investigated and found:

Unbounded divergence is rare
Divergence *can* happen, but in deep RL it is **not common** in practice.

Bootstrapping from separate networks reduces divergence
→ **Target networks** greatly stabilize learning.

Correcting overestimation helps
→ **Double DQN** mitigates the harmful maximization bias.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

What Happened to the Deadly Triad?

Additional stabilizing factors

Multi-step returns = less bootstrapping → less divergence
Higher *n*-step returns rely more on actual rewards.

Larger / more expressive networks diverge less
Bigger, more flexible networks behave more like tabular methods.

Stronger prioritization of updates (high ω) **will diverge more easily**, which is **bad**. **But then**, the amount of **update can be corrected via importance sampling** and that helps **to prevent divergence**.

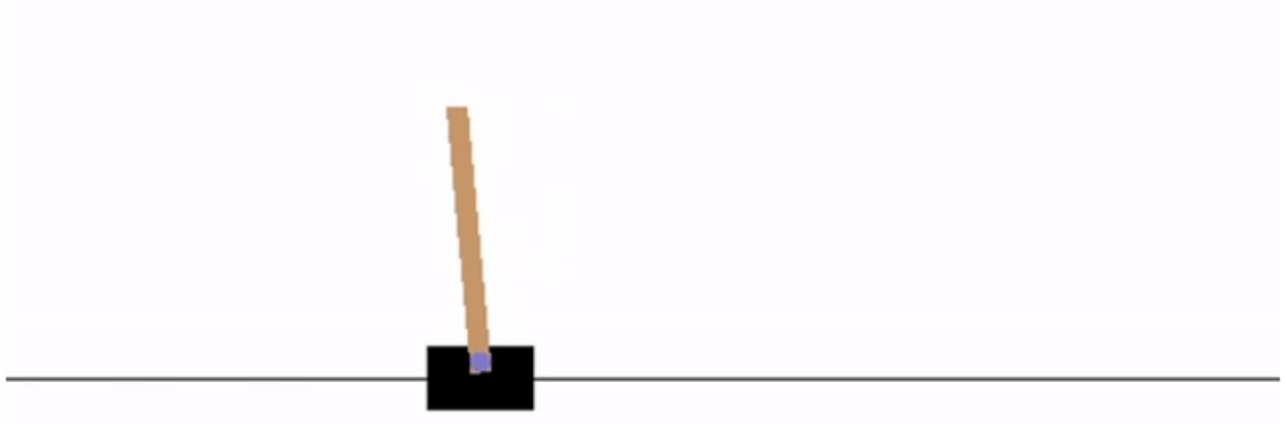
Takeaway

Deep Q-learning survives the deadly triad thanks to:
target networks, double Q-learning, multi-step returns, and sufficient model capacity, plus **importance sampling corrections** when using prioritized replay.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

DQN Tutorial

- https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html
- This tutorial shows how to use PyTorch to train a Deep Q Learning (DQN) agent on the CartPole-v1 task from Gymnasium.



Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."